Topic: Computer architecture

Objective: The objective is to balance the performance, efficiency, cost and reliability of computer system.

Outcome: Student will learn, how computer memory used to quickly accept, store and transfer data and instructions that are being used immediately by CPU.

What is computer architecture?

Computer architecture can be defined as a set of rules and methods that describe the functionality, management and implementation of computers. To be precise, it is nothing but rules by which a system performs and operates.

1.) Computer Architecture is concerned with the way hardware components are connected together to form a computer system.

2.) It acts as the interface between hardware and software.

3.) Computer Architecture helps us to understand the functionalities of a system.

4.) A programmer can view architecture in terms of instructions, addressing modes and registers.

5.) While designing a computer system architecture is considered first.

6.) Computer Architecture deals with high-level design issues.

7.) Architecture involves Logic (Instruction sets, Addressing modes, Data types, Cache optimization)

Why computer architecture?

Because, computer architecture is perhaps the most fundamental subject in computer science. Without computers, the field of computer science does not.

Sub-divisions

Computer Architecture can be divided into mainly three categories, which are as follows −

- **Instruction set Architecture or ISA** − Whenever an instruction is given to processor, its role is to read and act accordingly. It allocates memory to instructions and also acts upon memory address mode (Direct Addressing mode or Indirect Addressing mode).
- **Micro Architecture** − It describes how a particular processor will handle and implement instructions from ISA.
- **System design** − It includes the other entire hardware component within the system such as virtualization, multiprocessing.

Role of computer Architecture

The main role of Computer Architecture is to balance the performance, efficiency, cost and reliability of a computer system.

For Example − Instruction set architecture (ISA) acts as a bridge between computer's software and hardware. It works as a programmer's view of a machine.

Computers can only understand binary language (i.e., 0, 1) and users understand high level language (i.e., if else, while, conditions, etc). So to communicate between user and computer, Instruction set Architecture plays a major role here, translating high level language to binary language.

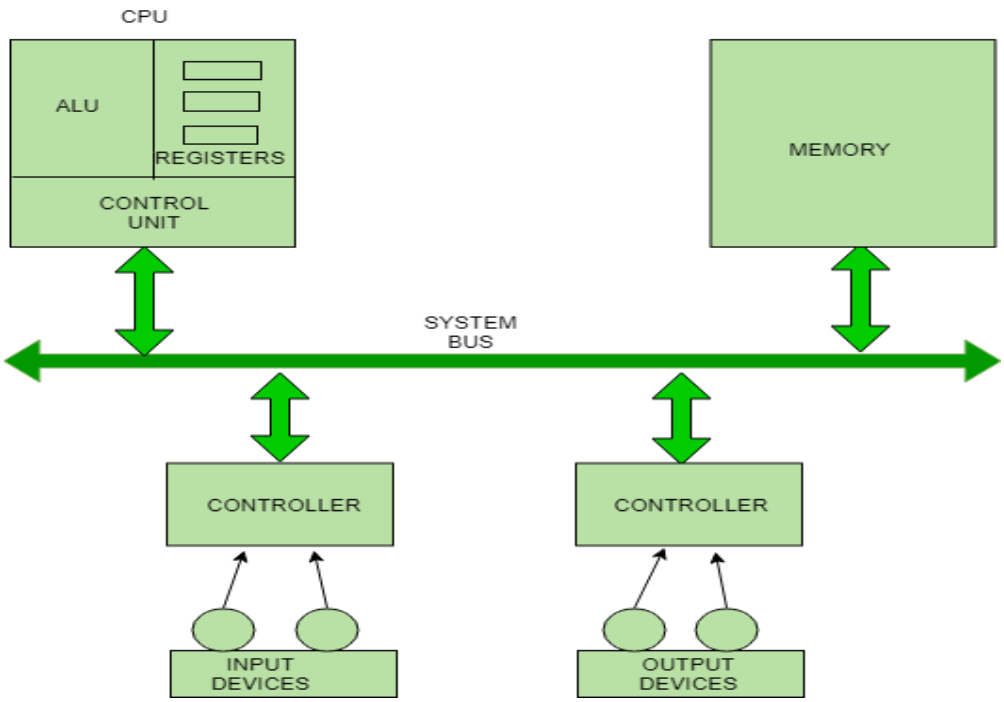
Structure

Let us see the example structure of Computer Architecture as given below.

Generally, computer architecture consists of the following −

- Processor
- Memory
- Peripherals

The diagram given below depicts the computer architecture −



The different components in the Computer System Architecture are Input Unit, Output Unit, Storage Unit, Arithmetic Logic Unit, Control Unit etc.

Details about all the computer units are −

- **Input Unit**
  The input unit provides data to the computer system from the outside. So, basically it links the external environment with the computer. It takes data from the input devices, converts it into machine language and then loads it into the computer system. Keyboard, mouse etc. are the most commonly used input devices.

- **Output Unit**
  The output unit provides the results of computer process to the users i.e it links the computer with the external environment. Most of the output data is the form of audio or video. The different output devices are monitors, printers, speakers, headphones etc.
- **Storage Unit**
  Storage unit contains many computer components that are used to store data. It is traditionally divided into primary storage and secondary storage.Primary storage is also known as the main memory and is the memory directly accessible by the CPU. Secondary or external storage is not directly accessible by the CPU. The data from secondary storage needs to be brought into the primary storage before the CPU can use it. Secondary storage contains a large amount of data permanently.
- **Arithmetic Logic Unit**
  All the calculations related to the computer system are performed by the arithmetic logic unit. It can perform operations like addition, subtraction, multiplication, division etc. The control unit transfers data from storage unit to arithmetic logic unit when calculations need to be performed. The arithmetic logic unit and the control unit together form the central processing unit.
- **Control Unit**
  This unit controls all the other units of the computer system and so is known as its central nervous system. It transfers data throughout the computer as required including from storage unit to central processing unit and vice versa. The control unit also dictates how the memory, input output devices, arithmetic logic unit etc. should behave

SUMMARY - With computers being more technology-based, computer architecture will continue to be an interesting career for candidates interested in developing these machines.

Topic: Register Transfer Language

Objective: This programming language is a procedure for writing symbols to specify a given computational process.

Outcome: Student will learn to describe the internal organization of digital computers in concise and precise manner.

Register Transfer Language

The symbolic notation used to describe the micro operation transfers among register is called a register transfer language.
A register transfer language is a system for expressing in symbolic form the micro operation sequences among the register of a digital module.

The term **register transfer** means the availability of **hardware logic circuits** that can perform a stated micro-operation and transfer the result of the operation to the same or another register.

The **Register Transfer Language** is the symbolic representation of notations used to specify the sequence of micro-operations.

In a computer system, data transfer takes place between processor registers and memory and between processor registers and input-output systems. These data transfer can be represented by standard notations given below:

- Notations R0, R1, R2..., and so on represent processor registers.
- The addresses of memory locations are represented by names such as LOC, PLACE, MEM, etc.
- Input-output registers are represented by names such as DATA IN, DATA OUT and so on.
- The content of register or memory location is denoted by placing square brackets around the name of the register or memory location.

Following are some commonly used registers:

1. **Accumulator**: This is the most common register, used to store data taken out from the memory.
2. **General Purpose Registers**: This is used to store data intermediate results during program execution. It can be accessed via assembly programming.
3. **Special Purpose Registers**: Users do not access these registers. These registers are for Computer system,

- **MAR:** Memory Address Register are those registers that holds the address for memory unit.
- **MBR:** Memory Buffer Register stores instruction and data received from the memory and sent from the memory.
- **PC:** Program Counter points to the next instruction to be executed.

- **IR:** Instruction Register holds the instruction to be executed.

The term “Register Transfer” can perform micro-operations and transfer the result of operation to the same or other register.

Micro-operations :
The operation executed on the data store in registers are called micro-operations. They are detailed low-level instructions used in some designs to implement complex machine instructions.

Register Transfer :
The information transformed from one register to another register is represented in symbolic form by replacement operator is called Register Transfer.

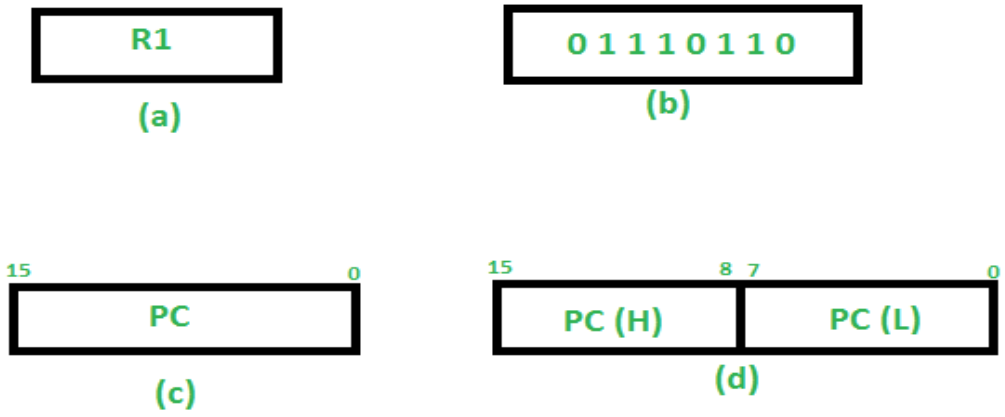
Replacement Operator :
In the statement, R2 <- R1, <- acts as a replacement operator. This statement defines the transfer of content of register R1 into register R2.
There are various methods of RTL –

1. General way of representing a register is by the name of the register enclosed in a rectangular box as shown in (a).

2. Register is numbered in a sequence of 0 to (n-1) as shown in (b).

3. The numbering of bits in a register can be marked on the top of the box as shown in (c).

4. A 16-bit register PC is divided into 2 parts- Bits (0 to 7) are assigned with lower byte of 16-bit address and bits (8 to 15) are assigned with higher bytes of 16-bit address



Basic symbols of RTL :

Symbol	Description	Example
Letters and Numbers	Denotes a Register	MAR, R1, R2
( )	Denotes a part of register	R1(8-bit) R1(0-7)
<-	Denotes a transfer of information	R2 <- R1
,	Specify two micro-operations of Register Transfer	R1 <- R2 R2 <- R1
:	Denotes conditional operations	P : R2 <- R1 if P=1
Naming Operator (:=)	Denotes another name for an already existing register/alias	Ra := R1

Register Transfer Operations:

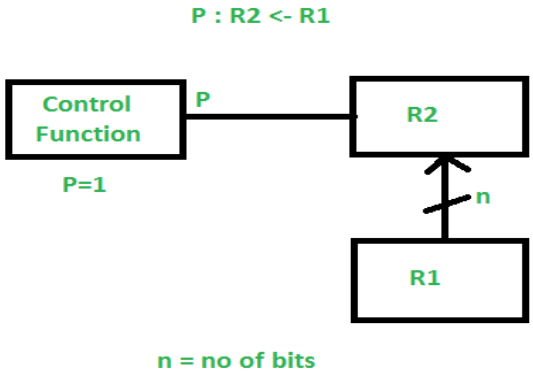
The operation performed on the data stored in the registers are referred to as register transfer operations.

There are different types of register transfer operations:

1. Simple Transfer – R2 <- R1

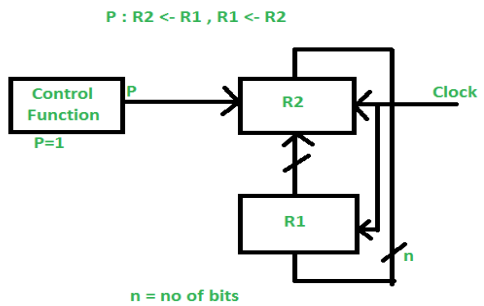
The content of R1 are copied into R2 without affecting the content of R1. It is an unconditional type of transfer operation.

2. Conditional Transfer –



It indicates that if P=1, then the content of R1 is transferred to R2. It is a unidirectional operation.

3. Simultaneous Operations –
If 2 or more operations are to occur simultaneously then they are separated with comma **(,)**.



If the control function P=1, then load the content of R1 into R2 and at the same clock load the content of R2 into R1.

What is Bus Transfer in Computer Architecture?

A bus transfer is the most effective method to send data by using a common bus system. It is constructed using common bus registers in multiple registers. The mechanism of the bus includes a collection of lines. These lines are registers of one bit each, which share only one information at a time. The data transfer is contained by the control signals.

The two methods that can be used in Bus transfer are as follows −

- Using multiplexer
- Using three states bus buffers

Common Bus System

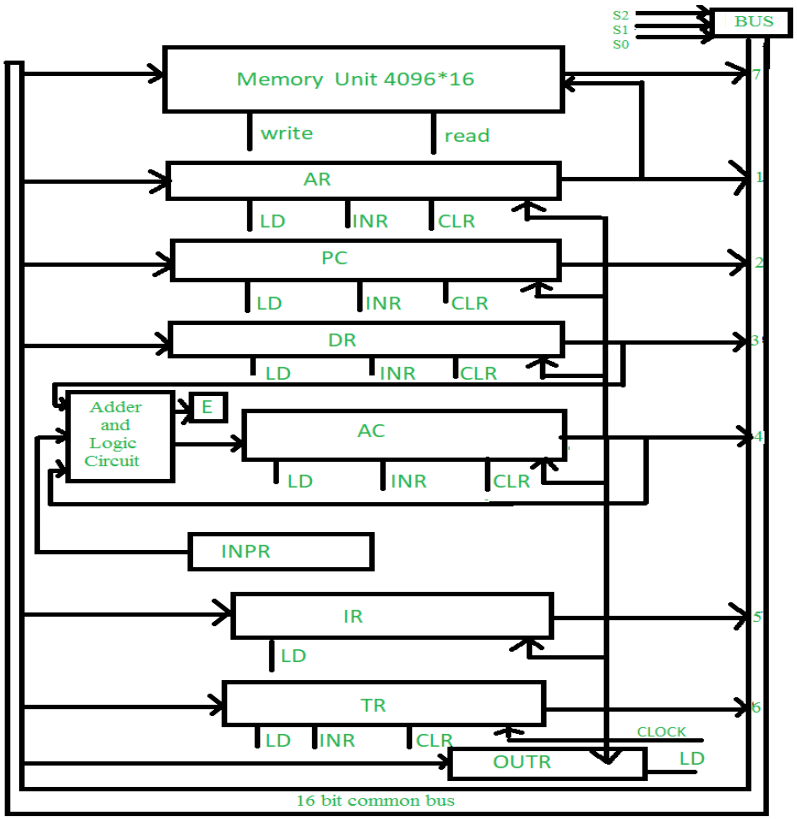
- **The basic computer has eight registers**, a memory unit, and a control unit . Paths must be provided to transfer information from one register to another and between memory and registers.
- **The number of wires will be excessive** if connections are made between the outputs of each register and the inputs of the other registers.
- **A more efficient** scheme for transferring information in a system with many registers is to use a common bus.
- **The connection of the registers** and memory of the basic computer to a common bus system is shown in Fig. below. The outputs of seven registers and memory are connected to the common bus.
- **The specific output** that is selected for the bus lines at any given time is determined from the binary value of the selection variables S_2, S_1, and S_0.
- **The number along** each output shows the decimal equivalent of the required binary selection. For example, the number along the output of DR is 3.

- **The 16-bit outputs of DR** are placed on the bus lines when $S_2S_1S_0$ = 011 since this is the binary value of decimal 3.

- **The lines from the common bus** are connected to the inputs of each register and the data inputs of the memory. The particular register whose LD (load) input is enabled receives the data from the bus during the next clock pulse transition.

- **The memory** receives the contents of the bus when its write input is activated. The memory places its 16-bit output onto the bus when the read input is activated and $S_2S_1S_0$ = 111.

- **Four registers**, DR, AC, IR, and TR, have 16 bits each.



Data movement from/to memory

Data movement instructions move data from one place, called the source operand, to another place, called the destination operand. Data movement instructions can be grouped into loads, stores, moves, and immediate loads.

- Load instructions move data from memory to registers.
- Store instructions move data from registers to memory.
- Move instructions move data from one register to another.
- Immediate load instructions move constants, including addresses, to registers.

The table shows the eight data transfer instructions and their respective mnemonic symbols.

Data Transfer Instructions

Name	Mnemonic Symbols
Load	LD
Store	ST
Move	MOV
Exchange	XCH
Input	In
Output	OUT
Push	PUSH
Pop	POP

Load Instructions

Load instructions move data from memory to registers.

Instruction	Operation
lb R_d, addr	load byte sign extended
lbu R_d, addr	load byte zero extended
lh R_d, addr	load halfword sign extended
lhu R_d, addr	load halfword zero extended
lw R_d, addr	load word
l.d F_d, addr	load double
l.s F_d, addr	load float

The instructions can be described as follows −

- **Load** − The load instruction is used to transfer data from the memory to a processor register, which is usually an accumulator.
- **Store** − The store instruction transfers data from processor registers to memory.
- **Move** − The move instruction transfers data from processor register to memory or memory to processor register or between processor registers itself.
- **Exchange** − The exchange instruction swaps information either between two registers or between a register and a memory word.
- **Input** − The input instruction transfers data between the processor register and the input terminal.
- **Output** − The output instruction transfers data between the processor register and the output terminal.
- **Push and Pop** − The push and pop instructions transfer data between a processor register and memory stack.

Examples include: **the transfer of vector operands in and out of vector registers**, the transfer of scalar operands in and out of a high-speed scalar processor, the movement of data between main memory and a high-speed cache or local memory, and paging between actual memory and disk storage in a virtual memory system.

SUMMARY - RTL is a simple, human-oriented language to specify the operations, register communication and timing of the steps that take place within a CPU to carry out higher level (user programmable) instructions.

Topic – Arithmetic & Logic unit & Control unit

Objective – It deals with the study of numbers using various operations on them.

Outcome - Help to do arithmetic and logic operations, including bit shifting operations.

ARITHMETIC & LOGIC UNIT

The **Arithmetic Logic Unit (ALU)** is the heart of any CPU. An ALU performs three kinds of operations, i.e.

- Arithmetic operations such as Addition/Subtraction,
- Logical operations such as AND, OR, etc. and
- Data movement operations such as Load and Store

How does an arithmetic-logic unit work?

Typically, the ALU has direct input and output access to the processor controller, main memory (random access memory or RAM in a personal computer) and input/output devices. Inputs and outputs flow along an electronic path that is called a bus.

The input consists of an instruction word, sometimes called a machine instruction word, that contains an operation code or "opcode," one or more operands and sometimes a format code. The operation code tells the ALU what operation to perform and the operands are used in the operation.

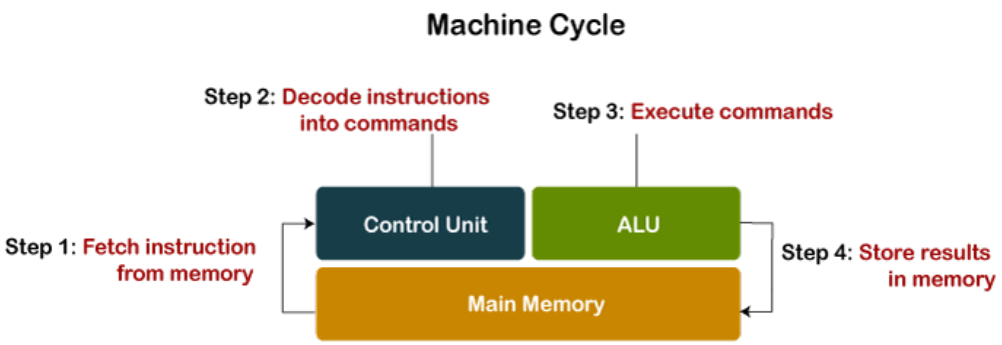
What type of functions do ALUs support?

In computer science, ALUs serve as a combinational digital circuit that performs arithmetic and bitwise operations on binary numbers. This is a foundational building block of arithmetic logic circuits for numerous types of control units and computing circuits including central processing units (CPUs), FPUs and graphics processing units.

The following are a few examples of bitwise logical operations and basic arithmetic operations supported by ALUs:

- **Addition.** Adds A and B with carry-in or carry-out sum at Y.
- **Subtraction.** Subtracts B from A or vice versa with the difference at Y and carry-in or carry-out.
- **Increment.** Where A or B is increased by one and Y represents the new value.
- **Decrement.** Where A or B is decreased by one and Y represents the new value.
- **AND.** The bitwise logic AND of A and B is represented by Y.

- **OR.** The bitwise logic OR of A and B is represented by Y.
- **Exclusive-OR.** The bitwise logic XOR of A and B is represented by Y.



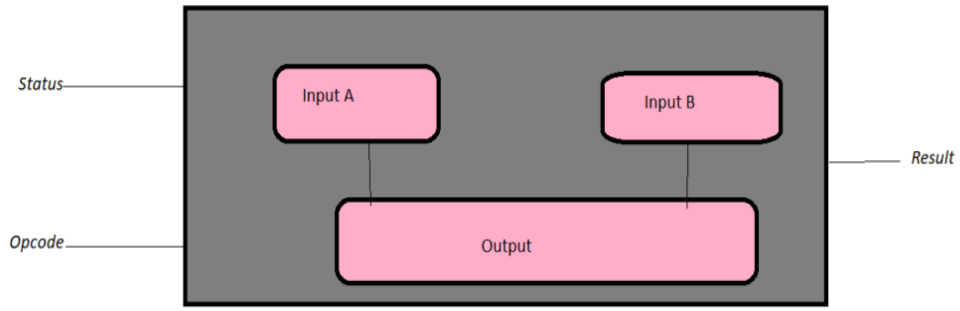
Arithmetic Logic Unit (ALU) Signals

Data: Three parallel buses are contained by the ALU, which include two input and output operand. These three buses handle the number of signals, which are the same.

Opcode: When the ALU is going to perform the operation, it is described by the operation selection code what type of operation an ALU is going to perform arithmetic or logic operation.

Status

- **Output:** The results of the ALU operations are provided by the status outputs in the form of supplemental data as they are multiple signals.
- **Input:** When ALU once performs the operation, the status inputs allow ALU to access further information to complete the operation successfully. Furthermore, stored carry-out from a previous ALU operation is known as a single "carry-in" bit.

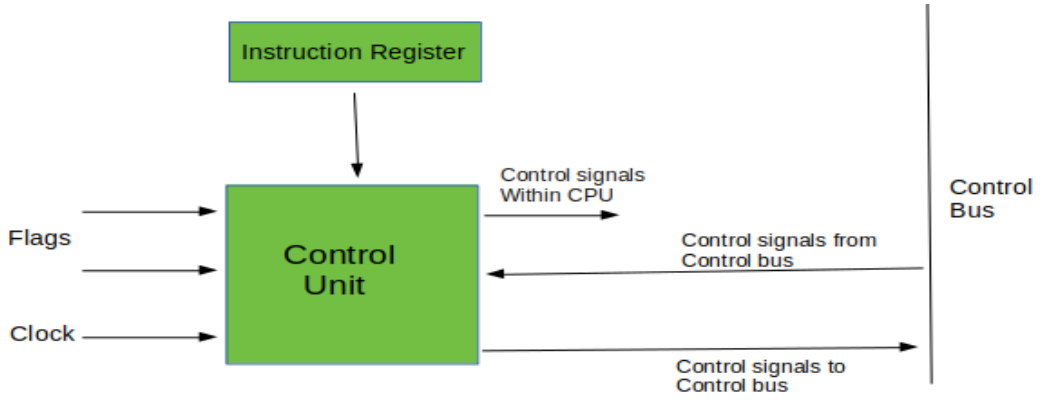


CONTROL UNIT

Control Unit is the part of the computer's central processing unit (CPU), which directs the operation of the processor.

A control unit works by receiving input information to which it converts into control signals, which are then sent to the central processor. The computer's processor then tells the attached hardware what operations to perform. The functions that a control unit performs are dependent on the type of CPU because the architecture of CPU varies from manufacturer to manufacturer. Examples of devices that require a CU are:

- Control Processing Units(CPUs)
- Graphics Processing Units(GPUs)



Block Diagram of the Control Unit

Functions of the Control Unit –

1. It coordinates the sequence of data movements into, out of, and between a processor's many sub-units.
2. It interprets instructions.
3. It controls data flow inside the processor.
4. It receives external instructions or commands to which it converts to sequence of control signals.
5. It controls many execution units(i.e. ALU, data buffers and registers) contained within a CPU.
6. It also handles multiple tasks, such as fetching, decoding, execution handling and storing results.

Summary - The ALU performs simple addition, subtraction, multiplication, division, and logic operations, such as OR and AND. The memory stores the program's instructions and data. The control unit fetches data and instructions from memory. A modern CPU has a very powerful ALU and it is complex in design. In addition to ALU modern CPU contains a control unit and a set of registers.

Topic - Synchronous & Asynchronous data transfer

Objective - The purpose of the study was to assess e-language learning analytics based on the constructivist approach of collaborative construction of knowledge.

Outcome – Students can diminish the difference between Synchronous & Asynchronous data transfer

What is Transmission?

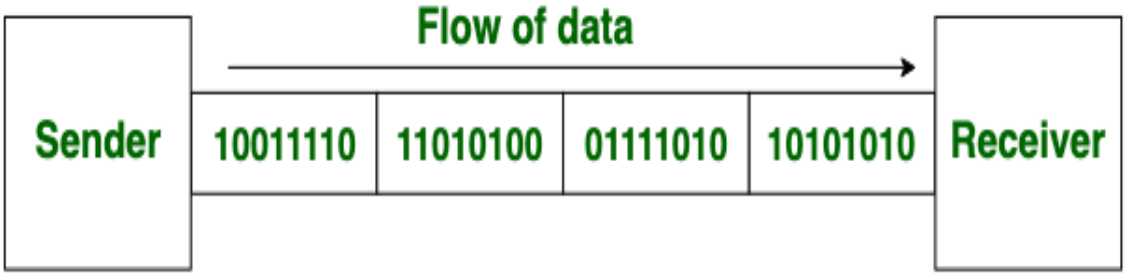
Transmission is the action of transferring or moving something from one position or person to another. It is a mechanism of transferring data between two devices connected using a network. It is also called communication Mode.

In computer networking there are two types of Transmission:

- Synchronous
- Asynchronous transmissions

What is Synchronous Transmission?

Synchronous data transmission is a data transfer method in which is a continuous stream of data signals accompanied by timing signals. It helps to ensure that the transmitter and the receiver are synchronized with each other.
This communication methods is mostly used when large amounts of data needs to be transferred from one location to the other.
In synchronous data transmission, data moves in a completely paired approach, in the form of chunks or frames. The synchronisation between the source and target is required so that the source knows where the new byte begins since there are no spaces included between the data.
Synchronous transmission is effective, dependable, and often utilised for transmitting a large amount of data. It offers real-time communication between linked devices.



Synchronous Transmission

In a digital system, if the other registers share the same clock with the CPU registers, the data transferring between the CPU and input and output devices is a synchronous data transfer. Both these units obtain the clock pluses from the common pulse generator.

How Does Synchronous Transmission work?

- Separate clocking lines used when the distance between the data terminal equipment (DTE) and data communications equipment (DCE) is short.
- This method uses a clocking electrical system at both transmitting and receiving stations. This ensures that the communication process is synchronized.
- Devices that communicate with each other Synchronously use either separate clocking channels.

Characteristics of Synchronous Transmission

- There are no spaces in between characters being sent.
- Timing is provided by modems or other devices at the end of the transmission.
- Special 'syn' characters goes before the data being sent.
- The syn characters are included between chunks of data for timing functions.

Examples of Synchronous Transmission

- Chatrooms
- Video conferencing
- Telephonic conversations
- Face-to-face interactions

Advantages of Synchronous Transmission

- It helps you to transfer a large amount of data.
- It offers real-time communication between connected devices.
- Each byte is transmitted without a gap between the next byte.
- It also reduces time timing errors.

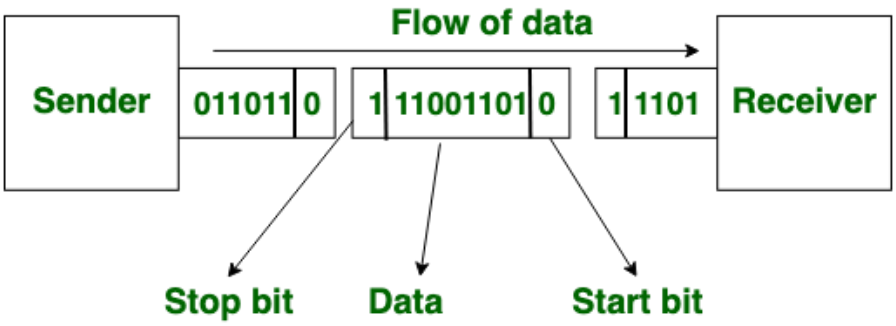
Disadvantages of Synchronous Transmission

- The accuracy of the received data depends on the receiver's ability to count the received bits accurately.
- The transmitter and receiver need to operate simultaneously with the same clock frequency.

What is Asynchronous Transmission?

- Asynchronous Transmission is also known as start/stop transmission, sends data from the sender to the receiver using the flow control method. It does not use a clock to synchronize data between the source and destination.
- This transmission method sends one character or 8 bits at a time. In this method, before the transmission process begins, each character sends the start bit. After sending the character, it also sends the stop bit. With the character bits and start and stop bits, the total number of bits is 10 bits.

In Asynchronous Transmission, data is sent in form of byte or character. This transmission is the half-duplex type transmission. In this transmission start bits and stop bits are added with data. It does not require synchronization.



Asynchronous Transmission

How Does Asynchronous Transmission work?

- Asynchronous communication is eased by two bits, which is known as start bit as ‘0’ and stop bit as ‘1.’
- You need to send ‘0’ bit to start the communication & ‘1’ bit to stop the Transmission.
- There is a time delay between the communication of two bytes.
- The transmitter and receiver may be function at different clock frequencies.

Characteristics of Asynchronous Transmission

- Each character is headed by a beginning bit and concluded with one or more end bits.
- There may be gaps or spaces in between characters.

Examples of Asynchronous Transmission

- Emails
- Forums
- Letters
- Radios
- Televisions

Advantages of Asynchronous Transmission

- This is a highly flexible method of data transmission.
- Synchronization between the receiver and transmitter is unnecessary.
- It helps you to transmit signals from the sources which have different bit rates.
- The Transmission can resume as soon as the data byte transmission is available.
- This mode of Transmission is easy for implementation.

Disadvantages of ASynchronous Transmission

- In Asynchronous Transmission, additional bits called start and stop bits are required to be used.
- The timing error may take place as it is difficult to determine synchronicity.
- It has a slower transmission rate.
- May create false recognition of these bits because of noise on the channel.

Synchronous and Asynchronous Transmission

Point of Comparison	**Synchronous Transmission**	**Asynchronous Transmission**
Definition	Transmits data in the form of chunks or frames	Transmits 1 byte or character at a time
Speed of Transmission	Quick	Slow
Cost	Expensive	Cost-effective
Time Interval	Constant	Random
Gaps between the data?	Does not exist	Exist
Examples	Chat Rooms, Telephonic Conversations, Video Conferencing	Email, Forums, Letters

Conclusion

The main difference between synchronous and asynchronous data transfer is that in synchronous data transfer, the transmitter and the receiver are synchronized with the same clock pulse while in asynchronous data transfer, the transmitter and the receiver do not use a common timing signal.

Summary - Synchronous transmission requires carefully synchronized clocks for the transmission of data, whereas asynchronous transmissions do not require synchronized clocks because the parity bits convey the information about data bytes.

Topic - Data Transfer Mode

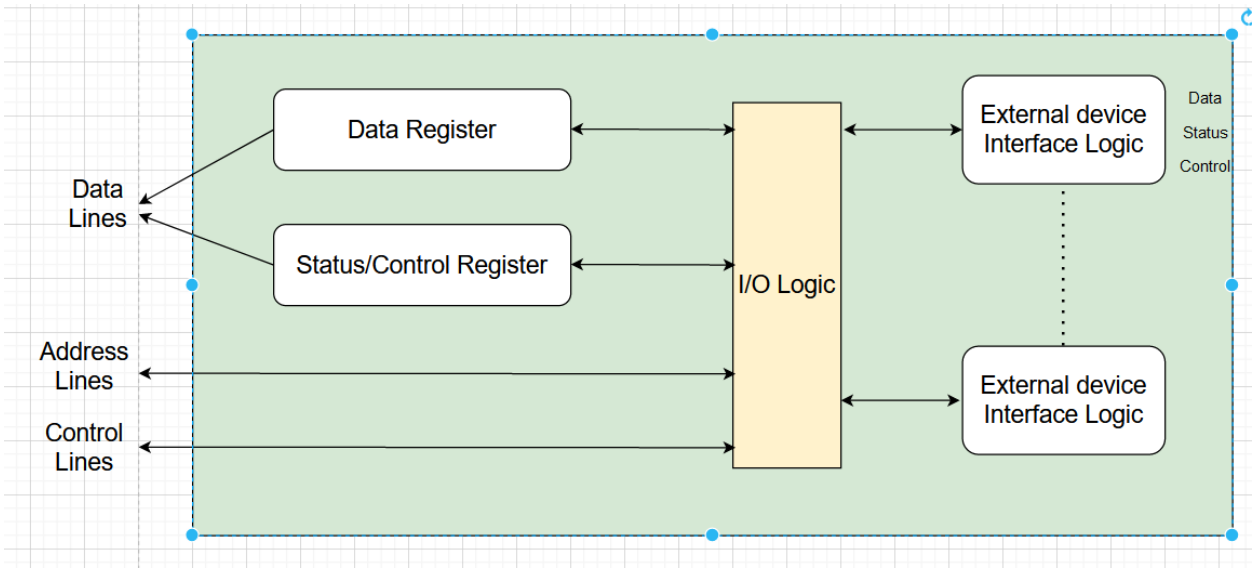
Objective – To learn transfer information between internal storage and external I/O devices

Outcome - It can provide an efficient means of transmitting data between the processing unit and the I/O devices.

Modes of Transfer

We store the binary information received through an external device in the memory unit. The information transferred from the CPU to external devices originates from the memory unit. Although the CPU processes the data, the target and source are always the memory unit. We can transfer this information using three different modes of transfer.

- The binary information that is received from an external device is usually stored in the memory unit.
- The information that is transferred from the CPU to the external device is originated from the memory unit.
- CPU merely processes the information but the source and target is always the memory unit.
- Data transfer between CPU and the I/O devices may be done in different modes.



We can transfer this information using three different modes of transfer.

1. Programmed I/O
2. Interrupt- initiated I/O
3. Direct memory access(DMA)

1.) Programmed I/O

Programmed I/O uses the I/O instructions written in the computer program. The instructions in the program initiate every data item transfer. Usually, the data transfer is from a memory and CPU register. This case requires constant monitoring by the peripheral device's CPU.

Advantages:

- Programmed I/O is simple to implement.
- It requires very little hardware support.
- CPU checks status bits periodically.

Disadvantages:

- The processor has to wait for a long time for the I/O module to be ready for either transmission or reception of data.
- The performance of the entire system is severely degraded.

2.) Interrupt-initiated I/O

In the above section, we saw that the CPU is kept busy unnecessarily. We can avoid this situation by using an interrupt-driven method for data transfer. The interrupt facilities and special commands inform the interface for issuing an interrupt request signal as soon as the data is available from any device. In the meantime, the CPU can execute other programs, and the interface will keep monitoring the i/O device. Whenever it determines that the device is ready for transferring data interface initiates an interrupt request signal to the CPU. As soon as the CPU detects an external interrupt signal, it stops the program it was already executing, branches to the service program to process the I/O transfer, and returns to the program it was initially running.

Working of CPU in terms of interrupts:

- CPU issues read command.
- It starts executing other programs.
- Check for interruptions at the end of each instruction cycle.
- On interruptions:-
    - Process interrupt by fetching data and storing it.
    - See operation system notes.
- Starts working on the program it was executing.

Advantages:

- It is faster and more efficient than Programmed I/O.
- It requires very little hardware support.
- CPU does not check status bits periodically.

Disadvantages:

- It can be tricky to implement if using a low-level language.
- It can be tough to get various pieces of work well together.
- The hardware manufacturer / OS maker usually implements it, e.g., Microsoft.

3.) Direct Memory Access (DMA)

The data transfer between any fast storage media like a memory unit and a magnetic disk gets limited with the speed of the CPU. Thus it will be best to allow the peripherals to directly communicate with the storage using the memory buses by removing the intervention of the CPU. This mode of transfer of data technique is known as Direct Memory Access (DMA). During Direct Memory Access, the CPU is idle and has no control over the memory buses. The DMA controller takes over the buses and directly manages data transfer between the memory unit and I/O devices.

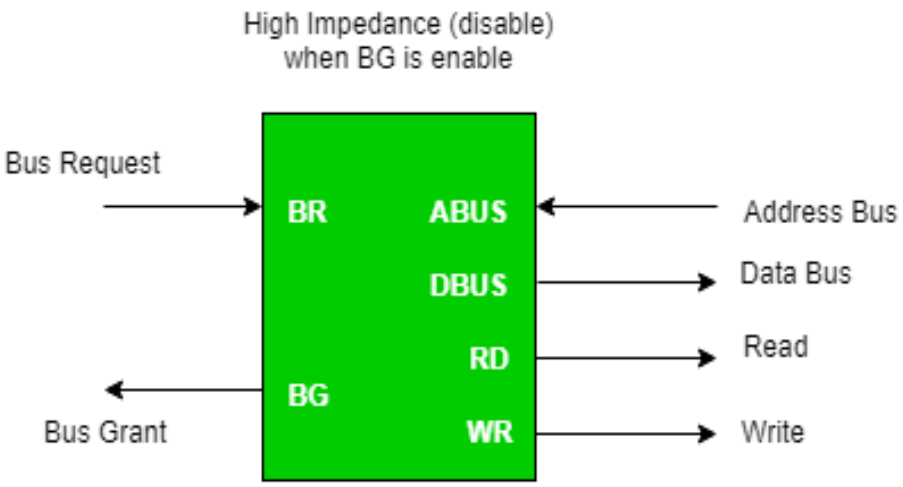


Figure - CPU Bus Signals for DMA Transfer

Bus Request - We use bus requests in the DMA controller to ask the CPU to relinquish the control buses.

Bus Grant - CPU activates bus grant to inform the DMA controller that DMA can take control of the control buses. Once the control is taken, it can transfer data in many ways.

Types of DMA transfer using DMA controller:

- **Burst Transfer:** In this transfer, DMA will return the bus control after the complete data transfer. A register is used as a byte count, which decrements for every byte transfer, and once it becomes zero, the DMA Controller will release the control bus. When the DMA Controller operates in burst mode, the CPU is halted for the duration of the data transfer.

- **Cyclic Stealing:** It is an alternative method for data transfer in which the DMA controller will transfer one word at a time. After that, it will return the control of the buses to the CPU. The CPU operation is only delayed for one memory cycle to allow the data transfer to "steal" one memory cycle.

Summary - Data transfer instructions transfer the data between memory and processor registers, processor registers, and I/O devices, and from one processor register to another. There are eight commonly used data transfer instructions. Each instruction is represented by a mnemonic symbol.

Topic – 8086 Microprocessor

Objective - It is responsible for transmitting address, data and controls signals on the buses.

Outcome - It consists of powerful instruction set, which provides operations like multiplication and division easily.

8086 Microprocessor

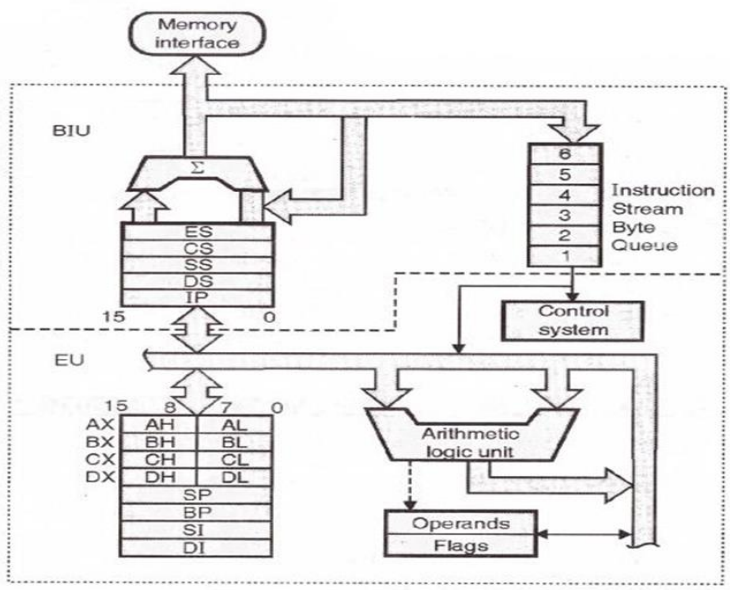
Definition: 8086 is a **16-bit microprocessor** and was created by Intel in 1978. Like the pin configuration of 8085 microprocessor, the 8086 microprocessor also contains **40 pins** dual in line.
However, unlike the 8085 microprocessor, an 8086 to have better performance, operates in 2 modes that are minimum and maximum mode.
The minimum mode is a single processor configuration while the maximum mode is a multiple processor configuration. Due to this reason, in the 40 pin IC of 8086 microprocessor, 8 pins i.e., pin numbered from 24 to 32 are assigned different configurations separately according to the two modes.

Features of 8086

The most prominent features of a 8086 microprocessor are as follows −

- It has an instruction queue, which is capable of storing six instruction bytes from the memory resulting in faster processing.
- It was the first 16-bit processor having 16-bit ALU, 16-bit registers, internal data bus, and 16-bit external data bus resulting in faster processing.
- It is available in 3 versions based on the frequency of operation −
  - 8086 → 5MHz
  - 8086-2 → 8MHz
  - (c)8086-1 → 10 MHz
- It uses two stages of pipelining, i.e. Fetch Stage and Execute Stage, which improves performance.
- Fetch stage can prefetch up to 6 bytes of instructions and stores them in the queue.
- Execute stage executes these instructions.
- It has 256 vectored interrupts.
- It consists of 29,000 transistors.

Architecture of 8086 Microprocessor Diagram



8086 internal architecture

These instructions are sent in the Control Unit for execution. The control unit takes the help of General Purpose registers, Index registers and Pointers, operands, flags and the most important, the Arithmetic Logic Unit (ALU). All these are part of the Execution Unit.

Bus Interface Unit (BIU)

1) Bus Interface Unit (BIU)

The Instruction Queue contains the set of instruction which is to be executed. To make the processing faster, the 8086 pre-fetches up to 6 instructions in advance and stores them in the Instruction queue. So, whenever one instruction completes its execution, the control unit need not wait for the next instruction to be fetched and then brought for execution because this job is already done and the next instruction that is to be executed is ready in the Instruction queue.

2) Segment Registers

Each Segment register can work with 16 bits of binary data. There are 4 types of segment registers:

i. CS: Code Segment Register
ii. DS: Data Segment Register
iii. SS: Stack Segment Register
iv. ES: Extra Segment Register

3) Instruction Pointer

The Instruction pointer contains the address of the next instruction that is to be executed.

Execution Unit (EU)

1) Control Unit (CU)

All the Instructions are executed inside the Control Unit. It is the main component which is responsible for the processing of any processor.

2) Arithmetic Logic Unit (ALU)

All the Mathematical and Logical Operations are performed inside the ALU. So, if any instruction needs to perform such operation, the Control Unit handovers it to the ALU.

3) Flag Register

The flag Register is of 16-bit length which consists of 9 flags, and the rest 7 bits are of don't care cases.

4) General Purpose Registers

The General Purpose Registers are used as containers for storing the values which may be required for executing the instructions. Each General Purpose Register consists of 16 bits. There are 4 types of General Purpose Registers:

i. AX = [AH:AL]
ii. BX = [BH:BL]
iii. CX = [CH:CL]

iv. DX = [DH:DL]

5) Pointers and Index Registers

The 8086 contains following Pointers and Index Registers. (Here, IP is not mentioned because it is a part of the BIU)

i. BP: Base Pointer
ii. SP: Stack Pointer
iii. SI: Source Index
iv. DI: Destination Index

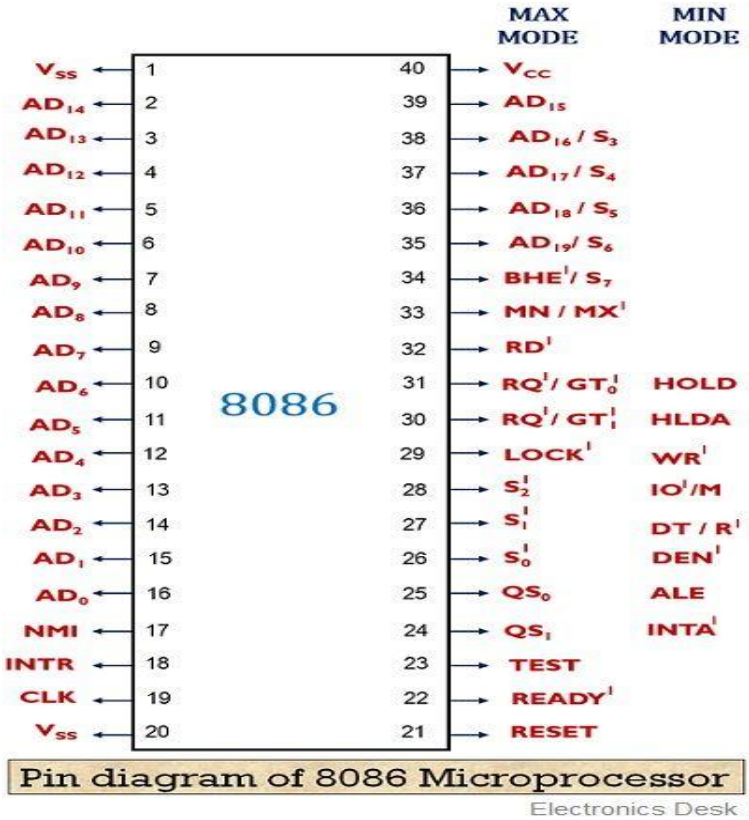
6) Operands

These may be used within the instructions.

Pin Diagram 8086 Microprocessor

An 8086 microprocessor is also a 40 pin IC but has few separate pin configuration for minimum and maximum mode which we will discuss in this article.

The figure below represents the pin diagram of 8086 microprocessor:



Pin diagram of 8086 Microprocessor
Electronics Desk

Now let us see the Pin functions of the 8086 microprocessor.

Pins	Function
AD_{15} – AD_0	These are 16 address/data bus. AD0-AD7 carries low order byte data and AD8AD15 carries higher order byte data. During the first clock cycle, it carries 16-bit address and after that it carries 16-bit data.
A_{16} – A_{19}	These are the 4 address/status buses. During the first clock cycle, it carries 4-bit address and later it carries status signals.
V_{cc}	It uses 5V DC supply at V_{CC} pin 40
GND	These are ground at V_{SS} pin 1 and 20.
Clock	Clock signal is provided through Pin-19. It provides timing to the processor for operations. Its frequency is different for different versions, i.e. 5MHz, 8MHz and 10MHz.
S7/BHE	BHE stands for Bus High Enable. It is available at pin 34 and used to indicate the transfer of data using data bus D8-D15. This signal is low during the first clock cycle, thereafter it is active.
Read(RD)	It is available at pin 32 and is used to read signal for Read operation.
Ready	It is available at pin 32. It is an acknowledgement signal from I/O devices that data is transferred. It is an active high signal. When it is high, it indicates that the device is ready to transfer data. When it is low, it indicates wait state.
RESET	It is available at pin 21 and is used to restart the execution. It causes the processor to immediately terminate its present activity. This signal is active high for the first 4 clock cycles to RESET the microprocessor.
INTR	It is available at pin 18. It is an interrupt request signal, which is sampled during the last clock cycle of each instruction to determine if the processor considered this as an interrupt or not.
NMI	It stands for non-maskable interrupt and is available at pin 17. It is an edge triggered input, which causes an interrupt request to the microprocessor.
TEST	This signal is like wait state and is available at pin 23. When this signal is high, then the processor has to wait for IDLE state, else the execution continues.
MN/MX	It stands for Minimum/Maximum and is available at pin 33. It indicates what mode the processor is to operate in; when it is high, it works in the minimum mode and vice-versa.
INTA	It is an interrupt acknowledgement signal and id available at pin 24. When the microprocessor receives this signal, it acknowledges the interrupt.
ALE	It stands for address enable latch and is available at pin 25. A positive pulse is generated each time the processor begins any operation. This signal indicates the availability of a valid address on the address/data lines.

Pins	Function
DEN	It stands for Data Enable and is available at pin 26. It is used to enable Transreceiver 8286. The transreceiver is a device used to separate data from the address/data bus.
DT/R	It stands for Data Transmit/Receive signal and is available at pin 27. It decides the direction of data flow through the transreceiver. When it is high, data is transmitted out and vice-a-versa.
M/IO	This signal is used to distinguish between memory and I/O operations. When it is high, it indicates I/O operation and when it is low indicating the memory operation. It is available at pin 28.
WR	It stands for write signal and is available at pin 29. It is used to write the data into the memory or the output device depending on the status of M/IO signal.
HLDA	It stands for Hold Acknowledgement signal and is available at pin 30. This signal acknowledges the HOLD signal.
HOLD	This signal indicates to the processor that external devices are requesting to access the address/data buses. It is available at pin 31.
QS_1 and QS_0	These are queue status signals and are available at pin 24 and 25. These signals provide the status of instruction queue.
S_0, S_1, S_2	These are the status signals that provide the status of operation, which is used by the Bus Controller 8288 to generate memory & I/O control signals. These are available at pin 26, 27, and 28.
LOCK	When this signal is active, it indicates to the other processors not to ask the CPU to leave the system bus. It is activated using the LOCK prefix on any instruction and is available at pin 29.
RQ/GT_1 and RQ/GT_0	These are the Request/Grant signals used by the other processors requesting the CPU to release the system bus. When the signal is received by CPU, then it sends acknowledgment. RQ/GT_0 has a higher priority than RQ/GT_1.

There are some conditions for QS_0 and QS_1. These conditions are as follows.

QS_0	QS_1	Status
0	**0**	No operation
0	**1**	First byte of opcode from the queue
1	**0**	Empty the queue

QS_0	QS_1	Status
1	**1**	Subsequent byte from the queue

The S2, S1 and S0 are the status signals. Here is a list of status and their bit patterns.

S_2	S_1	S_0	Status
0	**0**	**0**	Interrupt acknowledgement
0	**0**	**1**	I/O Read
0	**1**	**0**	I/O Write
0	**1**	**1**	Halt
1	**0**	**0**	Opcode fetch
1	**0**	**1**	Memory read
1	**1**	**0**	Memory write
1	**1**	**1**	Passive

Summary - 8086 has four 16-bit general purpose registers AX, BX, CX, and DX which store intermediate values during execution. Each of these has two 8-bit parts (higher and lower). It holds operands and results during multiplication and division operations.